

day12

反射机制

什么是java的反射机制

反射是java的动态机制，可以在【程序运行期间】再确定如:对象实例化，方法调用，属性操作等

- 反射可以提高代码的灵活性，可扩展性，但是带来了较多的系统开销和较慢的运行效率
- 反射机制不能被过度依赖

类对象

反射的第一步是获取一个类的类对象: `java.lang.Class`类

它的每一个实例被称为一个类的类对象

JVM每当需要使用一个类时就会加载它的.class文件(字节码文件)并同时实例化一个Class实例来记录加载的类的相关信息，并且每个被加载的类都有且只有一个Class的实例与之绑定，这个Class的实例就可以理解为是刚加载的类的类对象。

通过类对象可以反映出其表示的类的一切信息(类名，包信息，构造器信息，方法信息，属性信息)从而在程序运行期间进行动态调用

如何获取类对象

1. 类名.class
2. 对象.getClass()
3. Class.forName()
4. 类加载器ClassLoader

```
package reflect;

import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.Scanner;

/**
 * java反射机制
 * 反射是java的动态机制，可以在【程序运行期间】再确定如:对象实例化，方法调用，属性操作等
 * 反射可以提高代码的灵活性，可扩展性，但是带来了较多的系统开销和较慢的运行效率
 * 因此反射机制不能被过度依赖
 */
public class ReflectDemo1 {
    public static void main(String[] args) throws ClassNotFoundException {
```

```

        //获取String的类对象
//        Class cls = String.class;
//        Class cls = ArrayList.class;

        /*
            Class.forName(String className)获取类对象
            此时该方法要求传入一个字符串，表示对应类的完全限定名
        */
//        Class cls = Class.forName("java.lang.String");
//        Class cls = Class.forName("java.util.ArrayList");

        Scanner scanner = new Scanner(System.in);
        System.out.println("请输入一个类名: ");
        String cName = scanner.nextLine();
        Class cls = Class.forName(cName);

        //通过类对象反应出该类的名字
        String className = cls.getName();//获取完全限定名(包名.类名)
        System.out.println(className);
        className = cls.getSimpleName();//仅获取类名
        System.out.println(className);

    }
}

```

Package类

Package是反射对象之一，与Class类似，只不过它的每一个实例仅反应一个包的相关内容

获取包信息

Class提供的方法:

```
Package getPackage()
```

获取其表示的里的包信息。返回值为一个Package实例，该实例用于表示该包的信息

```

package reflect;

import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.Scanner;

/**
 * java反射机制
 * 反射是java的动态机制，可以在【程序运行期间】再确定如:对象实例化，方法调用，属性操作等

```

```

* 反射可以提高代码的灵活性，可扩展性，但是带来了较多的系统开销和较慢的运行效率
* 因此反射机制不能被过度依赖
*
*/
public class ReflectDemo1 {
    public static void main(String[] args) throws ClassNotFoundException {
        //获取String的类对象
        // Class cls = String.class;
        // Class cls = ArrayList.class;

        /*
            Class.forName(String className)获取类对象
            此时该方法要求传入一个字符串，表示对应类的完全限定名
        */
        // Class cls = Class.forName("java.lang.String");
        // Class cls = Class.forName("java.util.ArrayList");

        Scanner scanner = new Scanner(System.in);
        System.out.println("请输入一个类名：");
        String cName = scanner.nextLine();
        Class cls = Class.forName(cName);

        //通过类对象反应出该类的名字
        String className = cls.getName(); //获取完全限定名(包名.类名)
        System.out.println(className);
        className = cls.getSimpleName(); //仅获取类名
        System.out.println(className);

        /*
            获取包信息
            Class提供的方法：
            Package getPackage()
            获取其表示的里的包信息。返回值为一个Package实例，该实例用于表示该包的信息

            Package是反射对象之一，与Class类似，只不过它的每一个实例仅反应一个包的相关内容
        */
        Package pack = cls.getPackage();
        String packName = pack.getName(); //获取包名
        System.out.println("包名："+packName);
    }
}

```

反射机制实例化对象

Class提供的方法:instance可以使用无参构造器实例化

- 类必须提供公开且无参的构造器

- 无法抛出构造器实际抛出的异常，只能是:InstantiationException
- 在JDK9中不再建议使用

Person类代码

```
public class Person {

    private String name = "张三";
    private int age = 22;

    public Person() {}

    public Person(String name) {
        this.name = name;
    }

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    @Override
    public String toString() {
        return "Person{" +
            "name='" + name + '\'' +
            ", age=" + age +
            '}';
    }

}
```

```
package reflect;
```

```
/**
```

```

    * 使用反射机制实例化对象
    */
public class ReflectDemo2 {
    public static void main(String[] args) throws ClassNotFoundException,
    InstantiationException, IllegalAccessException {
        Person p = new Person();
        System.out.println(p);

        //1获取对应的类对象
        Class cls = Class.forName("reflect.Person");
        //2通过类对象的newInstance()方法调用该类的无参构造器实例化
        Object p2 = cls.newInstance();//new Person();
        System.out.println(p2);
    }
}

```

使用指定构造器实例化对象，Constructor类

Constructor类，反射对象之一 该类的每一个实例用于反应一个类中某个指定的构造器

通过构造器对象可以得知其表达的构造器的相关信息: 访问修饰符，参数列表等并且可以通过该构造器实例化对象

- 使用无参构造器实例化对象

```

package reflect;

import java.lang.reflect.Constructor;
import java.lang.reflect.InvocationTargetException;

/**
 * 使用指定的构造器实例化对象
 */
public class ReflectDemo3 {
    public static void main(String[] args) throws ClassNotFoundException,
    NoSuchMethodException, InvocationTargetException, InstantiationException,
    IllegalAccessException {
        //1加载类对象
        Class cls = Class.forName("reflect.Person");
        /*
        Constructor类，反射对象之一
        该类的每一个实例用于反应一个类中某个指定的构造器
        通过构造器对象可以得知其表达的构造器的相关信息：
        访问修饰符，参数列表等
        并且可以通过该构造器实例化对象
        */
        //2通过类对象反应它表示的类的特定构造器
        Constructor c = cls.getConstructor();//获取无参构造器
        Object o = c.newInstance();//new Person();
        System.out.println(o);
    }
}

```

```
}  
}
```

- 使用指定构造器实例化对象

```
package reflect;  
  
import java.lang.reflect.Constructor;  
import java.lang.reflect.InvocationTargetException;  
  
/**  
 * 使用指定的构造器实例化对象  
 */  
public class ReflectDemo3 {  
    public static void main(String[] args) throws ClassNotFoundException,  
        NoSuchMethodException, InvocationTargetException, InstantiationException,  
        IllegalAccessException {  
        //1加载类对象  
        Class cls = Class.forName("reflect.Person");  
        /*  
         * Constructor类，反射对象之一  
         * 该类的每一个实例用于反应一个类中某个指定的构造器  
         * 通过构造器对象可以得知其表达的构造器的相关信息：  
         * 访问修饰符，参数列表等  
         * 并且可以通过该构造器实例化对象  
         */  
        //2通过类对象反应它表示的类的特定构造器  
        Constructor c = cls.getConstructor(); //获取无参构造器  
        Object o = c.newInstance(); //new Person();  
        System.out.println(o);  
  
        /*  
         * 通过类对象的getConstructor()获取特定构造器时，需要在参数部分传入该构造器的参数列表  
         * 每个参数用对应的类对象即可。保证与实际构造器参数列表的顺序，个数，类型一致即可。  
         */  
        //Person(String,int)  
        Constructor c2 = cls.getConstructor(String.class, int.class);  
        //实例化时要传入实际参数  
        Object o2 = c2.newInstance("李四", 18); //new Person("李四", 18);  
        System.out.println(o2);  
  
        //Person(String)  
        Constructor c3 = cls.getConstructor(String.class);  
        Object o3 = c3.newInstance("王五");  
        System.out.println(o3);  
    }  
}
```

```
}
```

反射机制操作方法

获取一个类中的方法

Class提供了反应其表示的类的方法信息

```
Method[] getMethods()  
Method getMethod(String name, Class...)
```

Method类

Method，也是反射对象之一

它的每一个实例用于表示一个类中的某个方法通过Method对象可以反应出该方法的相关信息:

- 访问修饰符，返回值类型，方法名，参数信息等
- 可以通过方法对象执行这个方法

访问方法信息

在Person类上添加若干方法用于进行测试:

```
package reflect;  
  
import reflect.annotations.AutoRunClass;  
import reflect.annotations.AutoRunMethod;  
  
/**  
 * 使用当前类测试反射机制  
 */  
@AutoRunClass  
public class Person {  
  
    private String name = "张三";  
    private int age = 22;  
  
    public Person() {}  
  
    public Person(String name) {  
        this.name = name;  
    }  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
public void sayHello(){
    System.out.println(name+":hello!");
}

public void sayHi(){
    System.out.println(name+":hi!");
}

public void doSome(){
    System.out.println(name+":做某事");
}

public void sleep(){
    System.out.println(name+":在睡觉");
}

public void watchTV(){
    System.out.println(name+":看电视");
}

public void study(){
    System.out.println(name+":在学习");
}

public void playGame(){
    System.out.println(name+":在玩游戏");
}

public void sing(){
    System.out.println(name+":在唱歌");
}

public void say(String info){
    System.out.println(name+": "+info);
}

public void say(String info,int count){
    for (int i = 0; i < count; i++) {
        System.out.println(name+": "+info);
    }
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
```



```

        this.age = age;
    }

    @Override
    public String toString() {
        return "Person{" +
            "name='" + name + '\'' +
            ", age=" + age +
            '}';
    }
}

```

```

package reflect;

import java.lang.reflect.Method;
import java.lang.reflect.Modifier;
import java.util.Scanner;

/**
 * 获取一个类中的方法
 */
public class ReflectDemo4 {
    public static void main(String[] args) throws ClassNotFoundException {
        // 获取Person类中的方法信息
        Class cls = Class.forName("reflect.Person");

        Scanner scanner = new Scanner(System.in);
        System.out.println("请输入一个类名:");
        String className = scanner.nextLine();
        Class cls = Class.forName(className);

        // 通过类对象获取其表示的类中所有"公开"方法(包含从超类继承的方法)
        Method[] methods = cls.getMethods();
        for(Method method : methods){
            System.out.println(method);
            System.out.println("方法名:"+method.getName());
            // 获取该方法的参数个数
            int count = method.getParameterCount();
            System.out.println("参数的个数:"+count);
            // 获取该方法的访问修饰符
            int modifiers = method.getModifiers();
            switch (modifiers){
                case Modifier.PUBLIC:
                    System.out.println("公开方法");
                    break;
                case Modifier.PRIVATE:
                    System.out.println("私有方法");

```

```

        break;
    case Modifier.PROTECTED:
        System.out.println("受保护方法");
        break;
    }
}
}
}

```

调用方法

Method提供了用于调用其表示的方法的操作:

```
public Object invoke(Object obj, Object... args){...}
```

参数解释

- 参数1:该方法的所属对象
- 参数2:参数2开始可以传入若干参数，代表调用方法时的实参

```

package reflect;

import java.lang.reflect.Method;

/**
 * 反射机制调用方法
 */
public class ReflectDemo5 {
    public static void main(String[] args)throws Exception {
        Person p = new Person();
        p.sayHello();

        //1实例化对象
        Class cls = Class.forName("reflect.Person");
        Object o = cls.newInstance();
        //2调用方法
        //通过类对象获取Person的成员方法sayHello
        Method method = cls.getMethod("sayHello");
        //Method的重要方法:invoke()用于执行该方法，需要传入该方法的所属对象
        method.invoke(o);//p.sayHello
    }
}

```

暴力反射

像Field，Constructor，Method都提供了一个方法:

```
public void setAccessible(boolean flag){...}
```

该方法如果传入true，可以强行打开访问权限，用于访问本不可访问的内容。

比如私有方法是不能在类的外部访问的，但是通过调用对应的Method对象的setAccessible方法可以强制打开访问权限。

举例:

在Person类上添加私有方法hehe用于测试:

```
package reflect;

import reflect.annotations.AutoRunClass;
import reflect.annotations.AutoRunMethod;

/**
 * 使用当前类测试反射机制
 */
@AutoRunClass
public class Person {

    private String name = "张三";
    private int age = 22;

    public Person() {}

    public Person(String name) {
        this.name = name;
    }

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void sayHello() {
        System.out.println(name+":hello!");
    }

    public void sayHi() {
        System.out.println(name+":hi!");
    }

    public void doSome() {
        System.out.println(name+":做某事");
    }

    public void sleep() {
        System.out.println(name+":在睡觉");
    }

    public void watchTV() {
        System.out.println(name+":在看电视");
    }

    public void study() {
        System.out.println(name+":在学习");
    }
}
```

```
public void playGame(){
    System.out.println(name+":在玩游戏");
}
public void sing(){
    System.out.println(name+":在唱歌");
}

public void say(String info){
    System.out.println(name+": "+info);
}

public void say(String info,int count){
    for (int i = 0; i < count; i++) {
        System.out.println(name+": "+info);
    }
}

private void hehe(){
    System.out.println("我是Person的私有方法! ");
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}

@Override
public String toString() {
    return "Person{" +
        "name='" + name + '\'' +
        ", age=" + age +
        '}';
}
}
```

Class提供了一组:.getDeclared...方法, 都是用于获取本类定义的内容
Method getDeclaredMethod(String name...)
获取本类定义的某个方法
Method[] getDeclaredMethods()
获取本类定义的所有方法
上述两个方法获取的只有本类定义的方法, 含私有方法(不含超类继承的方法)

```
package reflect;

import java.lang.reflect.Method;

/**
 * 反射机制访问私有成员, 一般也称为暴力反射
 *
 * 该操作存在破坏类的封装性, 除非必要, 一般不要这样操作。
 */
public class ReflectDemo6 {
    public static void main(String[] args) throws Exception {
        //      Person p = new Person();
        //      p.hehe(); //编译不通过, 私有成员只能在类的内部被访问

        Class cls = Class.forName("reflect.Person");
        Object o = cls.newInstance();
        /*
            Class的方法:
            Method getMethod(String name...)
            Method[] getMethods()
            上述两个方法只能获取类的公开方法(含超类继承的)

            Class提供了一组:.getDeclared...方法, 都是用于获取本类定义的内容
            Method getDeclaredMethod(String name...)
            获取本类定义的某个方法
            Method[] getDeclaredMethods()
            获取本类定义的所有方法
            上述两个方法获取的只有本类定义的方法, 含私有方法(不含超类继承的方法)
        */

        //      Method[] methods = cls.getMethods();
        //      Method[] methods = cls.getDeclaredMethods();
        //      for(Method method : methods){
        //          System.out.println(method.getName());
        //      }

        //      Method method = cls.getMethod("hehe");
        Method method = cls.getDeclaredMethod("hehe");
        method.setAccessible(true); //强行打开访问权限
        method.invoke(o);
        method.setAccessible(false); //还原访问权限
    }
}
```

```
}  
}
```

调用有参方法

```
package reflect;  
  
import java.lang.reflect.InvocationTargetException;  
import java.lang.reflect.Method;  
  
/**  
 * 调用有参方法  
 */  
public class ReflectDemo7 {  
    public static void main(String[] args) throws ClassNotFoundException,  
InstantiationException, IllegalAccessException, NoSuchMethodException,  
InvocationTargetException {  
        Person p = new Person();  
        p.say("你好");  
  
        Class cls = Class.forName("reflect.Person");  
        Object o = cls.newInstance();  
        //say(String)  
        //第二个参数开始为参数列表  
        Method method = cls.getDeclaredMethod("say",String.class);  
        //调用方法时从第二个参数开始为方法的实际参数  
        method.invoke(o, "大家好");//p.say("大家好")  
  
        //say(String,int)  
        //p.say("嘿嘿",5)  
        Method method1 = cls.getDeclaredMethod("say",String.class,int.class);  
        method1.invoke(o, "嘿嘿",5);  
    }  
}
```

反射机制操作属性

Field类，它也是反射对象之一 它的每一个实例用于表示类中的某个属性

```
package reflect;  
  
import java.lang.reflect.Field;  
  
/**  
 * 反射机制操作方法  
 */  
public class ReflectDemo8 {  
    public static void main(String[] args) throws Exception {  
        Teacher teacher = new Teacher();  
        teacher.name = "王克晶";  
        System.out.println(teacher);  
    }  
}
```

```

class cls = Class.forName("reflect.Teacher");
Object o = cls.newInstance();
/*
    Field类，它也是反射对象之一
    它的每一个实例用于表示类中的某个属性
*/
//获取Teacher类的name属性
Field field =cls.getDeclaredField("name");
//    field.setAccessible(true);//如果是私有属性，可以强行访问
field.set(o,"范传奇");//o.name = "范传奇";
//    field.setAccessible(false);
System.out.println(o);

}
}

```

主流开发中操作属性都是通过调用属性的get,set方法进行，很少直接操作属性

什么是注解

注解 (Annotation) 是一种元数据 (metadata) 机制

可以使用注解来为代码中的各个部分添加额外的信息，以帮助程序的编译、运行或者其他处理过程。

注解的应用

Java注解可以用于:

- 为代码提供元数据信息，例如文档、版本号等
- 为编译器提供指示，例如抑制警告、生成代码等
- 为运行时提供指示，例如启用事务、配置参数等

注解功能一般都是使用Java反射API解析实现

Java注解为Java开发带来了很多好处，可以提高代码的可读性、可维护性和可靠性，从而使开发变得更加高效和轻松

注解的定义

```

public @interface AutoRunClass {
}

```

注解使用

注解可以在类的各个组成部分上使用，常见的位置:

- 类上使用
- 方法上使用
- 构造器上使用
- 属性上使用

- 方法参数上使用
- 在不限定注解使用位置时, 任何可以被应用的地方都可以使用注解(可以使用元注解@Target进行要求)

```
@AutoRunClass
public class Person {
    @AutoRunClass
    private String name = "张三";
    private int age = 22;

    @AutoRunClass
    public Person() {}

    public Person(@AutoRunClass String name){
        this.name = name;
    }

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @AutoRunClass
    public void sayHello(){
        System.out.println(name+":hello!");
    }
    ...
}
```

元注解

- @Target:用于标注当前注解可以被应用的位置.它的值对应枚举类:ElementType
 - ElementType.TYPE 类上, 接口上使用
 - ElementType.FIELD 在属性上使用
 - ElementType.METHOD 在方法上使用
 - ElementType.CONSTRUCTOR 在构造器上使用
 - 指定某个位置时:
@Target(ElementType.TYPE)
 - 同时指定多个位置时:
@Target({ElementType.TYPE, ElementType.FIELD})

- @Retention:表示当前注解的保留级别, 可选项是使用枚举RetentionPolicy表示的
- RetentionPolicy.SOURCE :当前注解近保留在源代码中, 编译后的字节码文件中没有该注解
- RetentionPolicy.CLASS :注解可以保留在字节码文件中, 但是不能为反射机制访问
- RetentionPolicy.RUNTIME :该注解保留在字节码文件中且可以被反射机制访问


```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface AutoRunClass {
}
```

反射机制访问注解

访问类上的注解

Person类上添加注解

```
package reflect;

import reflect.annotations.AutoRunClass;
import reflect.annotations.AutoRunMethod;

/**
 * 使用当前类测试反射机制
 */
@AutoRunClass
public class Person {

    private String name = "张三";
    private int age = 22;

    public Person() {}

    ...
}
```

所有反射对象都提供了一个方法：

```
boolean isAnnotationPresent(Class cls)
```

用于判断反射对象表示的内容是否被指定的注解标注
参数为一个类对象，表示指定的注解的类对象

常见的反射对象：

Class	它的每一个实例用于反映一个类的相关信息
Package	它的每一个实例用于反映一个包的信息
Method	每个实例反映一个方法的信息
Field	反映一个属性的信息
Constructor	反映一个构造器的信息
Parameter	反映一个参数的信息

```
package reflect;

import reflect.annotations.AutoRunClass;

/**
```

```

* 反射机制访问类上的注解
*/
public class ReflectDemo10 {
    public static void main(String[] args) throws ClassNotFoundException {
        //查看Person类上是否有注解@AutoRunClass

        //1:获取Person的类对象
        Class cls = Class.forName("reflect.Person");
        //2:通过类对象判断当前类是否被指定注解标注了
        boolean mark = cls.isAnnotationPresent(AutoRunClass.class);
        if(mark){
            System.out.println("被标注了!");
        }else{
            System.out.println("没有被标注!");
        }
    }
}

```

访问方法上的注解

在Person的方法上添加注解

```

package reflect;

import reflect.annotations.AutoRunClass;
import reflect.annotations.AutoRunMethod;

/**
 * 使用当前类测试反射机制
 */
@AutoRunClass
public class Person {

    private String name = "张三";
    private int age = 22;

    public Person() {}

    public Person(String name){
        this.name = name;
    }

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @AutoRunMethod(5)
    public void sayHello(){
        System.out.println(name+":hello!");
    }

    @AutoRunMethod(12)

```

```
public void sayHi(){
    System.out.println(name+":hi!");
}
public void doSome(){
    System.out.println(name+":做某事");
}
public void sleep(){
    System.out.println(name+":在睡觉");
}
@AutoRunMethod
public void watchTV(){
    System.out.println(name+":看电视");
}
public void study(){
    System.out.println(name+":在学习");
}
@AutoRunMethod
public void playGame(){
    System.out.println(name+":在玩游戏");
}
public void sing(){
    System.out.println(name+":在唱歌");
}

public void say(String info){
    System.out.println(name+": "+info);
}

public void say(String info,int count){
    for (int i = 0; i < count; i++) {
        System.out.println(name+": "+info);
    }
}

private void hehe(){
    System.out.println("我是Person的私有方法! ");
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}
```

```

    }

    @Override
    public String toString() {
        return "Person{" +
            "name='" + name + '\'' +
            ", age=" + age +
            '}';
    }
}

```

```

package reflect;

import reflect.annotations.AutoRunMethod;

import java.lang.reflect.Method;

/**
 * 反射机制访问方法上的注解
 */
public class ReflectDemo11 {
    public static void main(String[] args) throws ClassNotFoundException,
        NoSuchMethodException {
        //查看Person类的sayHello方法上是否有注解@AutoRunMethod
        Class cls = Class.forName("reflect.Person");
        Method method = cls.getDeclaredMethod("sayHello");

        boolean mark = method.isAnnotationPresent(AutoRunMethod.class);
        System.out.println("是否被注解@AutoRunMethod标注:"+mark);
    }
}

```

注解参数

注解可以指定参数，格式：

类型 参数名() [DEFAULT 默认值]

例如

```

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface AutoRunMethod {
    int value() default 1;
}

```

参数的定义与使用

- 使用注解传递参数时，使用的格式为:参数名=参数值

例如:

注解定义:

```
public @interface AutoRunMethod {  
    int age()  
}
```

当注解仅有一个参数,且参数名不为value时,正常使用注解传参语法:参数名=参数值

举例:

在Person类的方法sayHello上使用该注解,并指定参数:

Person部分代码展示:

```
@AutoRunMethod(age=3)                此时必须写"作参数名=参数值"  
public void sayHello(){  
    System.out.println(name+":hello!");  
}
```

```
@AutoRunMethod(3)                    编译不通过,因为参数没有指定参数名  
public void sayHello(){  
    System.out.println(name+":hello!");  
}
```

- 只有一个参数时，参数名应当选取value

例如:

如果注解仅有一个参数时,参数名使用value,则使用注解可以忽略参数名:

```
public @interface AutoRunMethod {  
    int value()  
}
```

使用时:

```
@AutoRunMethod(3)                    可以  
public void sayHello(){  
    System.out.println(name+":hello!");  
}
```

- 可以声明多个参数

注解可以声明多个参数

例如:

```
public @interface AutoRunMethod {  
    int age() default 1;  
    String name();  
}
```

当注解有多个参数时,使用该注解时每个参数都需要使用:参数名=参数值

例如:

```
@AutoRunMethod(age=2, name="张三")
public void sayHello(){
    System.out.println(name+":hello!");
}
```

实际使用中多个参数传参顺序可以与注解定义时参数顺序不一致

```
@AutoRunMethod(age=2, name="张三")
public void sayHello(){
    System.out.println(name+":hello!");
}
```

或

```
@AutoRunMethod(name="张三", age=2)
public void sayHello(){
    System.out.println(name+":hello!");
}
```

- 当注解有多个参数时,就算其中一个注解取名为value,实际使用时参数名也不可以忽略!

- 例如:

```
public @interface AutoRunMethod {
    int value();
    String name();
}
```

使用时:

@AutoRunMethod(name="张三", value=2)	可以
@AutoRunMethod(value=2, name="张三")	可以
@AutoRunMethod(name="张三", 2)	不可以
@AutoRunMethod(2, name="张三")	不可以

参数指定默认值,仍然在使用时可以忽略

```
public @interface AutoRunMethod {
    int value() default 1;
    String name();
}
```

@AutoRunMethod(name="张三")	可以
---------------------------	----

获取注解参数值

```
package reflect;

import reflect.annotations.AutoRunMethod;

import java.lang.reflect.Method;
```

```
/**
 * 访问注解参数
 */
public class ReflectDemo12 {
    public static void main(String[] args) throws ClassNotFoundException,
NoSuchMethodException {
        //获取Person类上sayHello方法上的注解参数值
        Class cls = Class.forName("reflect.Person");
        Method method = cls.getDeclaredMethod("sayHello");
        //判断方法上是否有注解:@AutoRunMethod
        if(method.isAnnotationPresent(AutoRunMethod.class)){
            /*
             所有反射对象都有一个方法:
             Annotation getAnnotation(Class cls)
             该方法可以获取对应的注解
            */
            //arm表示的就是sayHello方法上的@AutoRunMethod注解
            AutoRunMethod arm = method.getAnnotation(AutoRunMethod.class);
            //通过注解对象获取参数value的值
            int value = arm.value();
            System.out.println(value);
        }
    }
}
```